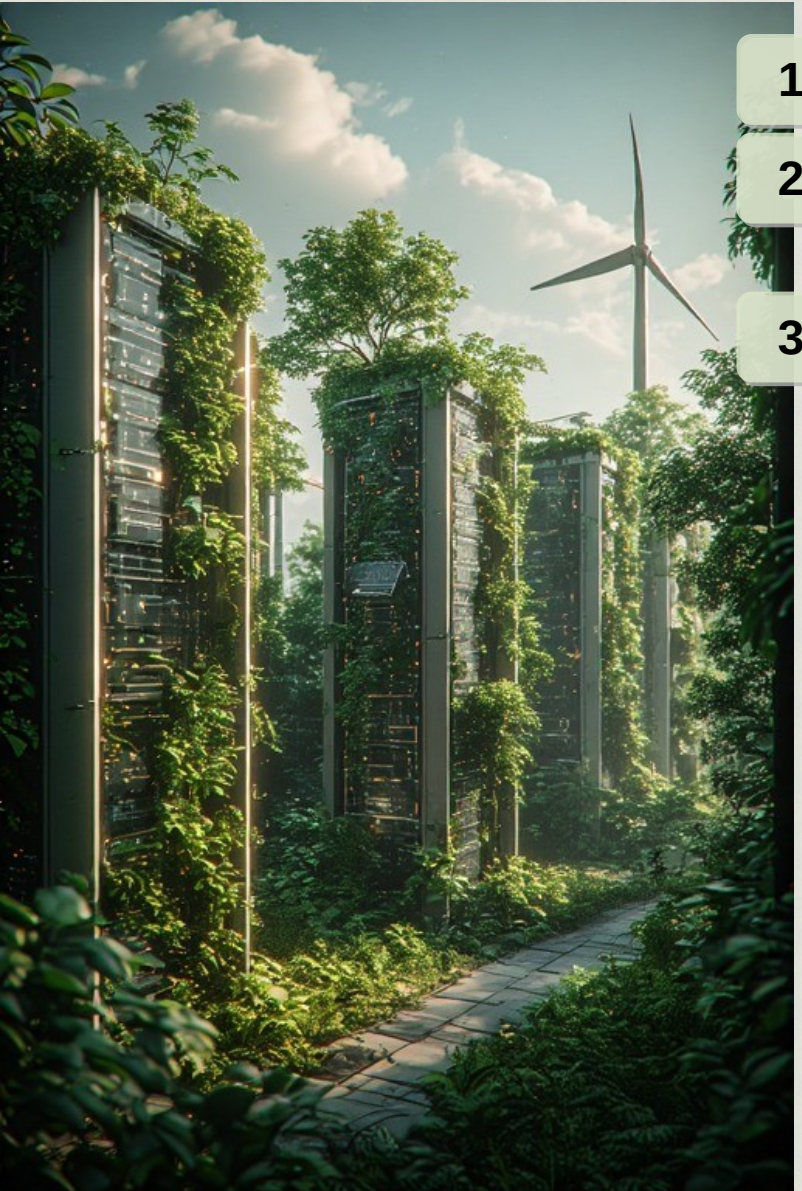
Sistemas Paralelos
Explicación práctica

Modelo de programación sobre memoria compartida (Pthreads - OpenMP)

Universidad Nacional de La Plata

Facultad de Informática

Prof. Adrian Pousa (apousa@lidi.info.unlp.edu.ar)



1 **Modelo de memoria compartida**

2 **Pthreads**

Estructura de programa. Definición de hilos. Creación, ejecución y finalización de hilos. Barreras y ejecución en etapas. Script de ejecución en cluster.

3 **OpenMP**

Script de ejecución en cluster.

1 Modelo de memoria compartida

Modelo de programación sobre memoria compartida

Una **variable compartida** es conceptualmente **diferente** a una **variable global**

- **Variable global** (**lenguajes imperativos): mecanismo de comunicación entre programa principal, módulos/subrutinas (procedures o funciones),** definidas en un área de programa visible por estos.
  - **Desaconsejado** en el desarrollo de **aplicaciones secuenciales** (por legibilidad, portabilidad, modularidad, mantenibilidad, etc.).
- **Variable compartida (modelo de memoria compartida): estado compartido** de cómputo concurrente entre hilos.
  - Son parte del modelo

```
Shared int x=0;
```

```
Process A{
    …
}
```

```
Process B{
    …
}
```

Confusión en lenguaje C:
No existe un área para definir variables compartidas y se definen como globales.

Una variable compartida NO es una variable global

Modelo de programación sobre memoria compartida

Un hilo no es una función o procedure

- Conceptualmente:
  - **Hilos no son funciones (Programación Concurrente)**
    - **No reciben** variables compartidas por **parámetros** (**porque son compartidas!!!**).
    - Ejemplo **ADA**: **Task** no son funciones ni procedures.
  - El código de los **hilos no** es **portable** sin el contexto del proceso que los contiene (modularidad SO), quien provee las variables compartidas.
  - Eventualmente, cada hilo tiene un **identificador único**.

```
Process P[id:0 to P-1]{
            …
}
```

Modelo de programación sobre memoria compartida

Imperativo (secuencial)

```
program Hello;
 var v: Integer;

procedure Proc()
begin
 v := v + 1;
end;

function f():integer
begin
 f:= v + 1;
end;
…
```

Pascal

```
#include …
 int v;

void f(){
 v++;
}
…
```

C

Imperativo
Esto está mal

POO
Esto está bien

```
Estructura de programa
  variable

 subrutina(sin parámetros)
  Modifica variable
```

POO (secuencial)

```
public class Clase{
 private int v;

 public void metodo(){
  v++;
 }
…
}
```

JAVA

```
Object subclass: Clase[
  |v|

 Clase » metodo()[
  v:= v + 1;
 ]
…
]
```

Smalltalk

Concurrente

```
int v;
Process P[id:0 to P-1]{
      atomic(v++);
      …
}
```

```
#include<pthread.h>
int v;

void* f(void* arg){
      atomic(v++);
      …
}

main(){
 …
 pthread_create(…,&f,…);
 …
}
```

C (Pthreads)

Concurrente
Esto está bien

En todos los casos, son estructuras de programa que definen una variable libre que se modifica por una subrutina directamente, sin parámetros. En imperativo está mal, en POO y Concurrente está bien. La **diferencia** está en el **paradigma (Imperativo ≠ POO ≠ Concurrente)**

2

Pthreads

Estructura de programa. Definición de hilos. Creación, ejecución y finalización de hilos. Barreras y ejecución en etapas. Script de ejecución en cluster.

Pthreads
Estructura de programa

- Programa **`main:`** proceso inicial.
  - Gestiona hilos
- Un programa Pthreads 4 pasos:
  1) Definir Hilos
  2) Crearlos
  3) Implementar la función de comportamiento del Hilo
  4) El proceso inicial espera que los hilos terminen su ejecución

```
#include<pthreads.h>
 //Variables compartidas
 //Mecanismos de sincronización

 //Función de comportamiento del hilo
 void* f(void* arg){
 …
  pthread_exit(NULL);
 }

int main(int argc, char*argv[]){
 //Definir un hilo
 pthread_t miHilo;
 …
 //Crear el hilo
 pthread_create(&miHilo,&attr,&f,&arg);
 …
 //Esperar que el hilo finalice
 pthread_join(&miHilo,NULL);

 return 0;
}
```

Pthreads
Definición de hilos

- Hilos representados mediante el tipo de datos:

```
pthread_t
```

- Pueden definirse individualmente:

```
pthread_t miHilo1;
pthread_t miHilo2;
```

Programación concurrente

```
Process P{
        …
}
```

- O como arreglos de hilos (varios hilos con el mismo comportamiento):

```
pthread_t misHilos[N];
```

Programación concurrente

```
Process P[id:0 to P-1]{
        …
}
```

Pthreads
Creación, ejecución y finalización de hilos

- Los hilos se crean y ponen en ejecución mediante la función:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);
```

No usar. Pasar NULL.

Si pthread_create tiene éxito, el hilo invoca automáticamente la función de su comportamiento.

```
void* funcion(void *arg){
int id=*(int*)arg;
      …
      pthreads_exit(NULL);
}

…
int main(int argc, char* argv[]){
pthread_t miHilo;
…
      id=1;
      pthread_create(&miHilo,NULL,&funcion,(void*)&id);
      pthreads_join(miHilo,NULL);
…
}
```

No retornar resultados.
Solo para control.
Siempre NULL en Memoria Compartida.

No pasar variables compartidas.
Solo pasar un id en Memoria Compartida.
(Clase 5 – Slide 21)

Pthreads
Barreras y ejecución en etapas

- **Aplicaciones con varias etapas de ejecución:**
  - **NO crear Hilos por cada etapa**
  - **Crear hilos una única vez y separar cada etapa por una barrera**

```
int main(int argc, char* argv[]){
…
 for(int id=0;id<T;id++){
    pthread_create(&h[id],NULL,&etapa1,(void*)id);
 }
…
 for(int id=0;id<T;id++){
    pthread_create(&h[id],NULL,&etapa2,(void*)id);
 }
…
}
```

```
void* etapas(void* arg){
      …
      etapa1();
      pthread_barrier_wait(&barrera);
      etapa2();
      pthread_barrier_wait(&barrera);
      …
}

int main(int argc, char* argv[]){
…
 for(int id=0;id<T;id++){
    pthread_create(&h[id],NULL,&etapas,(void*)id);
 }
…
}
```

Para el sistema operativo el costo de crear hilos una y otra vez es mayor que dormirlos y despertarlos

Pthreads
Características del script

```
#!/bin/bash
#SBATCH -N 1
#SBATCH –-exclusive
#SBATCH --partition=Blade
#SBATCH -o directorioSalida/output.txt
#SBATCH -e directorioSalida/errors.txt
#SBATCH --time=00:05:00 #Tiempo límite (HH:MM:SS)
./miAplicación parámetros de programa nroHilos
```

Idem secuencial

Aplicación Pthreads a ejecutar, sus respectivos parámetros y el número de hilos.

```
int main(int argc, char* argv[]){
//Se asume último parámetro
int T = atoi(argv[argc - 1]);
pthread_t hilos[T];

 for(int id=0;id<T;id++){
    pthread_create(…);
 }
…
}
```

```
chmod +x miScript.sh
```

Dar permisos de ejecución.

```
sbatch script
```

Ejecutar con el gestor de recursos. Retorna un JOBID

NO ejecutar así: `./script`

3

OpenMP

Script de ejecución en cluster.

OpenMP
Características del script

```
#!/bin/bash
#SBATCH -N 1
#SBATCH –-exclusive
#SBATCH --partition=Blade
#SBATCH -o directorioSalida/output.txt
#SBATCH -e directorioSalida/errors.txt
#SBATCH --time=00:05:00 #Tiempo límite (HH:MM:SS)
export OMP_NUM_THREADS = nro de hilos
./miAplicación parámetros de programa
```

Idem secuencial

El número de hilos por variable de entorno.
Aplicación OpenMP a ejecutar y sus respectivos parámetros.

Sin cambios en el código

`chmod +x miScript.sh` → Dar permisos de ejecución.

`sbatch script` → Ejecutar con el gestor de recursos. Retorna un JOBID

NO ejecutar así: `./script`

OpenMP
Características del script

```
#!/bin/bash
#SBATCH -N 1
#SBATCH –-exclusive
#SBATCH --partition=Blade
#SBATCH -o directorioSalida/output.txt
#SBATCH -e directorioSalida/errors.txt
#SBATCH --time=00:05:00 #Tiempo límite (HH:MM:SS)
./miAplicación parámetros de programa nroHilos
```

Idem secuencial

Aplicación OpenMP a ejecutar, sus respectivos parámetros y el número de hilos.

```
int main(int argc, char* argv[]){
//Se asume último parámetro
int T = atoi(argv[argc - 1]);
pthread_t hilos[T];

 #pragma omp parallel num_threads(T)
  {
      …
  }
}
```

```
chmod +x miScript.sh
```

Dar permisos de ejecución.

```
sbatch script
```

Ejecutar con el gestor de recursos. Retorna un JOBID

NO ejecutar así: `./script`